

```
"""
Lightweight SQLite storage for watched matches, odds snapshots, and
manually-tracked free-bet promos.
"""
```

```
import sqlite3
import time
from contextlib import contextmanager
```

```
DB_PATH = "oddsbot.db"
```

```
SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS watches (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sport_key TEXT NOT NULL,
    event_id TEXT NOT NULL,
    event_label TEXT NOT NULL,
    created_at INTEGER NOT NULL,
    UNIQUE(sport_key, event_id)
);
```

```
CREATE TABLE IF NOT EXISTS odds_snapshots (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id TEXT NOT NULL,
    outcome_name TEXT NOT NULL,
    price REAL NOT NULL,
    bookmaker TEXT NOT NULL,
    recorded_at INTEGER NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS free_bets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    bookmaker TEXT NOT NULL,
    description TEXT NOT NULL,
    amount TEXT,
    expires_at INTEGER,
    used INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL
);
```

```
"""
```

```
@contextmanager
def get_conn():
```

```

conn = sqlite3.connect(DB_PATH)
conn.row_factory = sqlite3.Row
try:
    yield conn
    conn.commit()
finally:
    conn.close()

def init_db():
    with get_conn() as conn:
        conn.executescript(SCHEMA)

# --- Watches ---

def add_watch(sport_key: str, event_id: str, event_label: str):
    with get_conn() as conn:
        conn.execute(
            "INSERT OR IGNORE INTO watches (sport_key, event_id, event_label, cre
            "VALUES (?, ?, ?, ?)",
            (sport_key, event_id, event_label, int(time.time()))),
        )

def list_watches() -> list[sqlite3.Row]:
    with get_conn() as conn:
        return conn.execute("SELECT * FROM watches ORDER BY created_at DESC").fet

def remove_watch(event_id: str):
    with get_conn() as conn:
        conn.execute("DELETE FROM watches WHERE event_id = ?", (event_id,))

# --- Odds snapshots (for line-movement detection) ---

def save_snapshot(event_id: str, outcome_name: str, price: float, bookmaker: str)
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO odds_snapshots (event_id, outcome_name, price, bookmaker
            "VALUES (?, ?, ?, ?, ?)",
            (event_id, outcome_name, price, bookmaker, int(time.time()))),
        )

def last_snapshot(event_id: str, outcome_name: str) -> sqlite3.Row | None:

```

```

with get_conn() as conn:
    return conn.execute(
        "SELECT * FROM odds_snapshots WHERE event_id = ? AND outcome_name = ?
        "ORDER BY recorded_at DESC LIMIT 1",
        (event_id, outcome_name),
    ).fetchone()

# --- Free bets (manual entry, since promo terms aren't structured data anywhere)

def add_free_bet(bookmaker: str, description: str, amount: str = "", expires_at:
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO free_bets (bookmaker, description, amount, expires_at, c
            "VALUES (?, ?, ?, ?, ?)",
            (bookmaker, description, amount, expires_at, int(time.time()))),
        )

def list_free_bets(include_used: bool = False) -> list[sqlite3.Row]:
    with get_conn() as conn:
        q = "SELECT * FROM free_bets"
        if not include_used:
            q += " WHERE used = 0"
        q += " ORDER BY expires_at ASC"
        return conn.execute(q).fetchall()

def mark_free_bet_used(bet_id: int):
    with get_conn() as conn:
        conn.execute("UPDATE free_bets SET used = 1 WHERE id = ?", (bet_id,))

def expiring_soon(within_seconds: int) -> list[sqlite3.Row]:
    now = int(time.time())
    with get_conn() as conn:
        return conn.execute(
            "SELECT * FROM free_bets WHERE used = 0 AND expires_at IS NOT NULL "
            "AND expires_at BETWEEN ? AND ?",
            (now, now + within_seconds),
        ).fetchall()

```